PAK-AUSTRIA FACHHOCHSCHULE:
INSTITUTE OF APPLIED SCIENCES AND TECHNOLOGY

Programming Fundamentals Lab 06	**Instructor: Mr. Muhammad Hamza**
Program: AI Fall 2025	**Date: 21th November, 2025**
Marks: 10	**Lab Engineer: Qandil Fatima**
Student Name: Eshal Ayub	**Reg. No: B25F1061AI100**

Instructions:

- Submit the file with your names following your registration numbers like **AI001_Name.**
- **Copy Paste work will be given no marks.**

LAB TASK

Task 1 : Password Input with Limited Attempts

Task Explanation:

In this task, the user must enter a password.
The system allows 3 attempts, and inside each attempt:

- **The password must be at least 8 characters long.**
- **If it is shorter, the program will repeatedly ask again using a while loop.**
- **Once a valid password is entered, the system accepts it.**

This exercise helps in understanding nested loops, input validation, and basic authentication logic.

Code:

[*] Untitled1 [*] Untitled2 [*] Untitled3

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    string password;

    for (int attempt = 1; attempt <= 3; attempt++) {
        cout << "Attempt " << attempt << ": Enter password: ";
        cin >> password;

        // Ask repeatedly until length >= 8
        while (password.length() < 8) {
            cout << "Password too short! Must be at least 8 characters.\n";
            cout << "Re-enter password: ";
            cin >> password;
        }

        cout << "Password accepted!\n";
        break;  // success ? exit the loop
    }

    return 0;
}
```

Output:

```
C:\Users\DELL PRECision 7550

Attempt 1: Enter password: ESHAAL123
Password accepted!

--------------------------------
Process exited after 37.1 seconds with return value 0
Press any key to continue . . .
```

```
Attempt 1: Enter password: 334
Password too short! Must be at least 8 characters.
Re-enter password: ESHAL123
Password accepted!

--------------------------------
Process exited after 15.14 seconds with return value 0
Press any key to continue . . .
```

Output Explanation:

- **If the user enters fewer than 8 characters, the program will immediately ask again.**
- **Once the password meets the condition, the message "Password accepted!" appears.**
- **If valid on the first attempt, remaining attempts are skipped.**

Task 2 : Teacher Enters Grades of N Students

Task Explanation:

The teacher must input grades for N students.

Rules:

- **Grade must be between 0 and 100.**
- **If the teacher enters an invalid number, the program asks again using a while loop.**

This task strengthens understanding of:

- **Input validation**
- **Repetition of input**
- **Looping over multiple students**

Code:

[*] Untitled1 [*] Untitled2 [*] Untitled3

```cpp
#include <iostream>
using namespace std;

int main() {
    int N, grade;

    cout << "Enter number of students: ";
    cin >> N;

    for (int i = 1; i <= N; i++) {
        cout << "Enter grade for student " << i << ": ";
        cin >> grade;

        while (grade < 0 || grade > 100) {
            cout << "Invalid! Grade must be 0 to 100.\n";
            cout << "Re-enter grade: ";
            cin >> grade;
        }

        cout << "Grade accepted!\n";
    }

    return 0;
}
```

Output:

```
C:\Users\DELL PRECision 7550

Enter number of students: 3
Enter grade for student 1: 90
Grade accepted!
Enter grade for student 2: 75
Grade accepted!
Enter grade for student 3: 100
Grade accepted!

--------------------------------
Process exited after 70.18 seconds with return value 0
Press any key to continue . . .
```

Output Explanation:

- **For each student, the program demands a valid grade.**
- **Numbers like -5, 200, or characters are rejected.**
- **Only 0–100 is accepted.**
- **After every correct entry, it prints "Grade accepted!"**

Task 3 : ATM PIN Simulation with 3 Attempts

Task Explanation

This task simulates how real ATMs work.

Rules:

1. **User gets 3 attempts to enter a PIN.**
2. **Inside each attempt:**
    - **PIN must be exactly 4 digits.**
3. **If PIN is not 4 digits → ask again.**
4. **If PIN is wrong → try again.**
5. **After 3 failed attempts → Card Blocked.**

This teaches:

- **Nested loops**
- **String validation**
- **Decision making**
- **Real-world simulation**

Code:

[*] Untitled1 [*] Untitled2 [*] Untitled3

```cpp
#include <string>
using namespace std;

int main() {
    string pin;
    string correctPin = "1234"; // example correct PIN

    for (int attempt = 1; attempt <= 3; attempt++) {
        cout << "Attempt " << attempt << ": Enter PIN: ";
        cin >> pin;

        // Ask until PIN is exactly 4 digits
        while (pin.length() != 4) {
            cout << "PIN must be exactly 4 digits!\n";
            cout << "Re-enter PIN: ";
            cin >> pin;
        }

        // Check correct PIN
        if (pin == correctPin) {
            cout << "Access granted!\n";
            return 0;
        }
        else {
            cout << "Incorrect PIN! Try again.\n";
        }
    }

    cout << "Card Blocked\n";
    return 0;
}
```

Output:

```
C:\Users\DELL PRECision 7550

Attempt 1: Enter PIN: 1234
Access granted!

--------------------------------
Process exited after 3.08 seconds with return value 0
Press any key to continue . . .
```

```
C:\Users\DELL PRECision 7550

Attempt 1: Enter PIN: 12
PIN must be exactly 4 digits!
Re-enter PIN: ABCD
Incorrect PIN! Try again.
Attempt 2: Enter PIN: 1111
Incorrect PIN! Try again.
Attempt 3: Enter PIN: 3333
Incorrect PIN! Try again.
Card Blocked

--------------------------------
Process exited after 67.03 seconds with return value 0
Press any key to continue . . .
```

Output Explanation

- **If the user mistypes the length, the program won't continue.**
- **If the PIN is incorrect, the user loses one attempt.**
- **After 3 wrong attempts → the system prints "Card Blocked", like a real ATM.**

---

5. Conclusion

In this lab we practiced:

- **Using for and while loops together**
- **Validating user input**
- **Creating real-life logic for passwords and ATM systems**
- **Ensuring the program continues until the correct input is given**
- **Implementing limited attempt systems**

These exercises build strong foundations for input validation and logical structuring in C++.